This use of methods on the left side of an assignment leads to a potential ambiguity between a local variable assignment and a method call:

```
class Person
  def name=(new_name)
    @name = new_name
  end

  def change_things(new_name, address)
    name = new_name
  end
end
```

In this example, it is potentially ambiguous whether the line `name = new_name` creates a local variable called `name` and assigns to it or whether it uses the implicit receiver syntax to call the method `self.name=`.

Ruby handles this potential issue consistently and always in favor of creating the local variable. When calling a method on the left side of an assignment, you must specify the receiver explicitly. In this case, the highlighted line must be changed to `self.name = new_name`. Not doing so can lead to some hard to track down bugs. (Ask us how we know.)

Passing Arguments to a Method

Any arguments follow the method name. If no ambiguity exists, you can omit the parentheses around the argument list when calling a method. Ruby documentation sometimes describes method calls without parentheses as *commands*, and as you've seen, method calls that look like commands or macros, such as `puts`, are often written without parentheses. There are also a lot of Ruby tools, like RSpec, that skip parentheses to make their domain specific languages flow more easily.

However, except in the simplest cases we don't recommend this in general—some subtle problems can trip you up. In particular, you *must* use parentheses on a method call that is itself an argument to another method call (unless it is the last parameter). Our rule is simple: if you have any doubt, use parentheses.

```
# for some suitable value in obj:
a = obj.hash    # Same as
a = obj.hash()  # this.

obj.some_method "Arg1", arg2, arg3   # Same thing as
obj.some_method("Arg1", arg2, arg3)  # with parentheses.
```

Positional arguments are passed to the method based on their position, but keyword arguments are passed based on the keyword, and can be listed in any order:

```
def method_with_keywords(city:, state:, zip:)
  "I live in #{city}, #{state} #{zip}"
end

puts method_with_keywords(city: "Chicago", state: "IL", zip: "60606")
puts method_with_keywords(zip: "02134", city: "Boston", state: "MA")
```

produces:

```
I live in Chicago, IL 60606
I live in Boston, MA 02134
```